

Module 2: Parallel Image Processing with Python

Author: Xiaoyuan Suo, Webster University

Target Courses: CS2 (Python), Data Science, Systems, Intro HPC

Prerequisite Module: Basics of Parallel Programming in Python (Module 1)

This module introduces data-parallel image processing as a motivating application of parallel and distributed computing (PDC). Building on students' prior exposure to threads, processes, and basic performance concepts, the module demonstrates how real-world computational workloads, such as image blurring, naturally expose issues of decomposition, data dependency, correctness, and scalability.

1. Moving Beyond Loops: Vectorization

While parallel programming (multiprocessing) focuses on doing multiple tasks at once across different CPU cores, Vectorization is about doing multiple things at once on a single core.

In a standard Python for loop, you are telling the CPU: "Take this one pixel, add 10 to it, and put it back. Now do the next one." This is slow because of Python's overhead. Vectorization allows you to say: "Take this entire row of 1,000 pixels and add 10 to all of them in one single clock cycle."

Feature	Parallel Programming (Multiprocessing)	Vectorization (SIMD)
Level	Multi-Core (High-level)	Single-Core Instruction (Low-level)
Analogy	Hiring 4 chefs to cook 4 different pizzas.	Using a special and very powerful machine to cook 4 pizzas at once. Or use a giant stamp like knife to cut 10 pepperoni slices at once
Python Tool	multiprocessing, joblib	numpy, pandas
Hardware	Multiple CPU Cores	SIMD Registers (Single Instruction, Multiple Data)
Example	<pre># Parallelism (multiple chefs) from multiprocessing import Pool def process_chunk(chunk): return chunk * 1.2 chunks = np.array_split(image, 4) with Pool(4) as p: result = p.map(process_chunk, chunks)</pre>	<pre># Vectorization (one chef, many pixels at once) import numpy as np image = np.random.rand(1000, 1000) brightened = image * 1.2 # applies to all pixels at once</pre>

Why do I teach vectorization?

If you teach students to use `multiprocessing` to loop through pixels, they will actually find it's often **slower** than sequential code because of the "Pickle" overhead (the time it takes Python to package data and send it to another process).

Vectorization via NumPy is usually the "correct" first step in Python optimization.

A Quick Comparison

If you want to brighten an image (add value to every pixel):

a. The "Naive" Way (Slow):

```
for x in range(width):
    for y in range(height):
        pixels[x, y] += 10
```

b. The Vectorized Way (NumPy - Fast):

```
# This happens at near-C speeds
pixels += 10
```

c. The Parallel Way (Multiprocessing - Scalable): Splitting the image into 4 quarters and giving one quarter to each CPU core to run the vectorized code.

```
import numpy as np
import multiprocessing as mp
from PIL import Image
import time

def process_shard(shard):
    """
    This function runs on a separate CPU core.
    It uses Vectorization (NumPy) to process the data at C-speed.
    """
    # Example: Simple brightness increase and color inversion
    # This is a 'Vectorized' operation: no for-loops!
    brightened = np.clip(shard.astype(np.int16) + 30, 0,
255).astype(np.uint8)
    return 255 - brightened

def parallel_image_processor(image_path, num_cores=4):
    # 1. Load image and convert to NumPy array
    img = Image.open(image_path).convert('RGB')
    data = np.array(img)

    # 2. Split the array into 'shards' (quarters) along the vertical
axis
    # array_split handles cases where the image height isn't perfectly
divisible
    shards = np.array_split(data, num_cores)
```

```

print(f"Starting parallel processing on {num_cores} cores...")
start_time = time.time()

# 3. Create a Process Pool
with mp.Pool(processes=num_cores) as pool:
    # Map the vectorized function to each shard
    processed_shards = pool.map(process_shard, shards)

# 4. Reassemble the shards back into one image
final_data = np.vstack(processed_shards)
final_img = Image.fromarray(final_data)

end_time = time.time()
print(f"Task completed in {end_time - start_time:.4f} seconds.")

final_img.show()
return final_img

if __name__ == "__main__":
    # Ensure you have an image named 'input.jpg' in your directory
    # Or replace with a valid path
    try:
        parallel_image_processor("input.jpg", num_cores=mp.cpu_count())
    except FileNotFoundError:
        print("Please provide a valid image path to test the code.")

```

The best lesson for students is showing them how to **combine both**.

- Vectorize the math within each task (using NumPy) so each core is as fast as possible.
- Parallelize those tasks across multiple cores (using Multiprocessing) to handle massive datasets.

2. Definitions and Core Concepts

This module deepens students' understanding of PDC by grounding abstract ideas in image processing pipelines, where correctness and performance are both visible and measurable.

Image Processing as a Data-Parallel Problem

- Images are represented as large 2D (or 3D) arrays of pixels.
- Many operations (filtering, convolution, color transforms) apply the same computation independently across pixels or regions.
- This makes image processing an ideal example of data parallelism.

Students are introduced to common image-processing tasks:

- Enhancement (e.g., blurring, denoising)
- Feature extraction (edges, patterns)
- Segmentation

Gaussian Blur and Stencil Computation

- Gaussian blur is a stencil operation: each output pixel depends on a neighborhood of input pixels.
- This introduces data dependency, distinguishing it from embarrassingly parallel tasks.
- The concept of a halo (ghost region) is introduced to ensure correctness when partitioning images.

This is a critical conceptual leap from Module 1, where summation had no spatial dependency.

Parallel Decomposition Strategies

Students explore:

- Horizontal vs. vertical image partitioning
- Chunk size selection
- Trade-offs between load balance and communication

3. Course Integration: Where This Module Fits

Suitable Courses

- CS2 (Python-based)
- Data Science / Analytics
- Systems Programming
- Computer Vision (intro)
- HPC or Performance Engineering electives

Integration with Existing Topics

Existing Topic	Parallel Image Processing Connection
Arrays / matrices	Images as NumPy arrays
Functions	Stateless worker functions
Classes	Encapsulating pipelines
Algorithms	Stencil patterns
Software testing	Correctness validation via diff images

This module fits naturally after students learn multiprocessing basics, providing a realistic use case.

4. Limitations

Possible pitfalls:

- There is no textbook—I made all the examples by myself.

- Incorrect results without halo regions
- False confidence from visually “almost correct” outputs
- Speedup saturation due to memory bandwidth
- High overhead from excessive process creation
- Misleading performance if OpenCV’s internal threading is not controlled

Students are required to demonstrate failure cases (no-halo mode) to reinforce correctness-first thinking.

Assignment Parallel Image blur (see assignment)

5. Pedagogical Notes for Instructors

For many single-image operations, Python image-processing code appears fast without explicit parallelism because vectorized libraries such as OpenCV perform the heavy computation in optimized, multithreaded C/C++. However, understanding explicit parallel decomposition remains essential for correctness, scalability, and multi-image or distributed workloads.

- Image processing is highly motivating: students *see* correctness and errors.
- Visual diff images make abstract correctness issues concrete.
- This module naturally bridges theory, programming, and performance evaluation.
- Python is used deliberately as a teaching language, while explicitly acknowledging that production systems often rely on C++/CUDA underneath.

What I have included in the slides and sample programs are explicit image partition. Explicit image partitioning is most valuable for teaching parallel concepts when it exposes decomposition, data dependency, and correctness issues; for production image processing, vectorized and library-managed parallelism is usually the better choice. Listed below are image processing tasks that are suitable for explicit image partition and some are not.

Some Image Processing Tasks Well-Suited for Explicit Image Partitioning

- **Gaussian blur / box blur / median filter**
These stencil-based operations naturally support spatial decomposition. Dividing an image into stripes or tiles (with halo regions) cleanly illustrates data dependency, correctness issues, and the cost of boundary handling, making them ideal for teaching parallel decomposition and ghost regions.
- **Edge detection (Sobel, Prewitt, Laplacian)**
Each output pixel depends on a small neighborhood, so explicit partitioning exposes halo size, correctness trade-offs, and performance overhead while remaining conceptually manageable for students.
- **Thresholding and pixel-wise transformations**
Operations such as grayscale conversion, thresholding, normalization, or color inversion are embarrassingly parallel and easy to split across regions, making them good introductory examples before introducing data dependencies.

- **Histogram computation (per-tile reduction)**
Computing partial histograms on image tiles followed by a reduction step closely mirrors parallel reduction patterns in HPC and reinforces concepts from earlier modules (e.g., parallel sum).
- **Processing batches of images**
Treating each image as an independent task is an excellent example of task parallelism and avoids halo complexity, while still motivating multiprocessing and scalability discussions.

Image Processing Tasks Best Left to Existing Vectorized / Library Methods (no explicit partition)

- **Single-call filtering using OpenCV (e.g., cv2.GaussianBlur)**
These functions are already implemented in optimized C/C++ and often use SIMD and multithreading internally. Re-parallelizing them in Python usually adds overhead without improving performance.
- **Color space conversions (BGR ↔ Gray, RGB ↔ HSV)**
These are highly optimized, memory-bound operations where Python-level parallelism offers little benefit and may degrade performance.
- **Geometric transformations (resize, rotate, warp)**
These operations rely on carefully optimized interpolation routines and memory access patterns that are difficult to reproduce efficiently at the Python level.
- **Feature detectors and descriptors (SIFT, ORB, FAST)**
These algorithms are complex, tightly optimized, and internally parallelized; attempting to partition them in Python is error-prone and offers minimal pedagogical gain.
- **Deep learning–based image processing**
CNN inference and training are already parallelized on GPUs or optimized CPU backends; explicit Python multiprocessing interferes with framework schedulers and is not recommended.

5. Included Materials

- Lecture slides: *Image Processing with Multiprocessing Using Python*

parallel image
- Assignment: *Parallel Gaussian Blur with Correctness and Performance Evidence*

Assignment Parallel Image blur
- Starter code for serial and parallel blur
- Benchmarking and plotting requirements
- Correctness validation via diff metrics and images